COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/17/2026	**DATE SUBMITTED:**	7/22/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 2 ARRAYS, POINTERS AND DYNAMIC MEMORY ALLOCATION

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

Screenshot

Procedure1.cpp

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student(std::string newName ="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

        //deconstructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }
        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
        //Display Attributes
        void printDetails(){
            std::cout << this->studentName << " " << this->studentAge << std::endl;
        }
    };

int main() {

    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}

```

Observation	 The program starts by loading the necessary libraries (iostream for basic input/output and string.h for std::string). It then defines a Student class where each Student object has two private data members. studentName (a string type) and studentAge (an integer type). These members are declared as private, so they won't be accessed directly outside the class (this restriction will help protect the object's data). The Student class has several special member functions. First we have the constructor, it initializes a new Student object by assigning a name (studentName = std::move(newName)) and age (studentAge = newAge). If no values are provided, it uses the default values "John Doe" for the name and 18 for the age. Every time the constructor runs, it prints "Constructor Called." to show that a new object has been created. Now for the destructor, where it automatically gets called when a Student object is destroyed. The destructor simply prints out "Destructor Called." to indicate that the object is being removed from memory, although it does not free any dynamic memory because it does not have delete ptr in this code. We have the copy constructor, this function is called whenever a new Student object is initialized using an existing Student object (provided from the code). It copies the name and age from the original object into the new object and prints "Copy Constructor Called" so that the user can observe when this type of copying occurs. The last function in the code is named printDetails(), which displays the student's name and age but even though this function was defined, it is never called inside the main() function, so no student information is printed during the program's execution.

Table 2-1. Initial Driver Program

Screenshot

Procedure2.cpp > ...

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student(std::string newName ="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

        //deconstructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }
        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
        //Display Attributes
        void printDetails(){
            std::cout << this->studentName << " " << this->studentAge << std::endl;
        }
    };

int main() {

    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

Observation

The program has all the same functions as the last one but the only thing that was changed was the contents of the int main() where instead of creating individual Student objects, it creates an array containing five Student objects. It also creates two additional arrays that store student names and ages.

std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};

However, these arrays are not used to initialize or update the Student objects, so every object in the studentList array retains its default values.

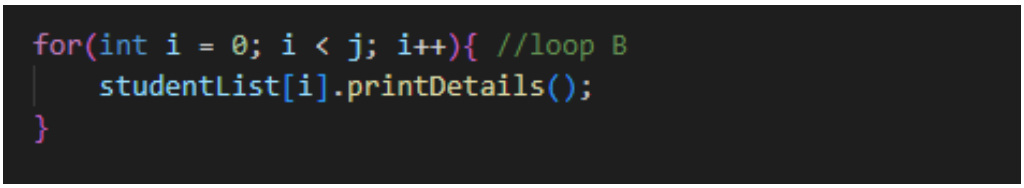
Inside our main() we have const size_t j = 5 which creates a constant variable named j that

	holds the value of 5. This will be used as the size of all the arrays coded in the program. From my observation, using a constant makes the program easier to modify because it will automatically change the size of every array in the program if we change the value of j to something else. Student studentList[j] = {}; will create an array named studentList that contains five Student objects. However, we don't have any name or age provided for any of the objects, so in this case each object is initialized using the constructor's default parameters (studentName = "John Doe" and studentAge = 18). As a result, the constructor is automatically called five times (because the value of j is 5), once for each object in the array. We do have std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"} but because they are simply names stored in memory. They are not assigned to any Student object because there is no code that copies these values into studentList. The same goes for the ageList array, they are not copied either into the Student objects. As a result, all five Student objects remain unchanged from their default state throughout execution. The program ends by automatically calling the destructor for each object, which shows that arrays of objects follow the same process as individual objects, with constructors running during creation and destructors running automatically when the objects go out of scope. `Constructor Called.` (×5) `Destructor Called.` (×5)

Table 2-2. Modified Driver Program with Student Lists

Loop A	`for(int i = 0; i < j; i++){ //loop A`
	`  Student *ptr = new Student(namesList[i], ageList[i]);`
	`  studentList[i] = *ptr;`
	`}`

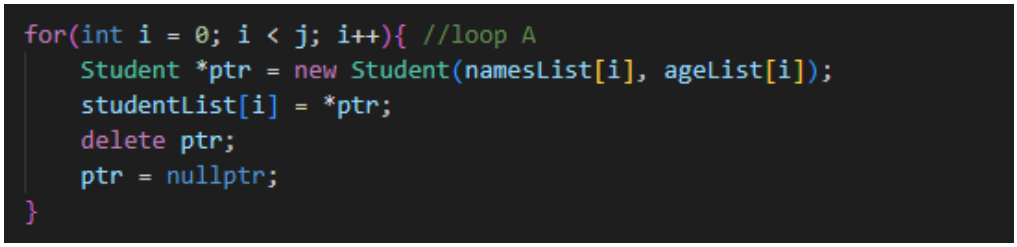
Observation	Loop A is responsible for assigning the names ("Carly", "Freddy", "Sam", "Zack", "Cody") and ages (15, 16, 18, 19, 16) stored in nameList and ageList to each Student object in the studentList array. The loop starts with i = 0 and continues until i = 4, since the array size j is 5 (as stated before). A temporary Student object is created at every iteration by using the constructor for that particular name and age in the arrays and then the temporary object is assigned to studentList[i] through the copy assignment operator then the object gets destroyed when the iteration ends then moves on to the next one.

Loop B	
Observation	Loop B is responsible for displaying the information stored in each Student object. The loop again starts at i = 0 and continues until i = 4. During each iteration, the printDetails() function is called for the current object.
Output	
Observation	When the program runs, the first output displayed is “Constructor Called.” is printed 5 times and this is because of the statement: Student studentList[j] = {}; where j has a value of 5 And what I’ve mentioned earlier, since no name or age is provided for these objects, the constructor will use the default values (being “John Doe” and 18). Loop A will make a temporary Student object is created at every iteration by using the constructor for that particular name and age in the arrays and then the temporary object is assigned to studentList[i] through the copy assignment operator then the object gets destroyed when the iteration ends then moves on to the next one. That is why we see alternating pattern of “Constructor Called” and “Destructor Called” Loop B will then display the names of each Student object in an organized matter, then when the program reaches the end of main() all the objects in the studentList array will be no longer in use, so the destructor is automatically called for each of the five Student objects. This is why we see “Destructor Called.” 5 times because it will destruct from studentList[4] all the way to studentList[0].

```
for(int i = 0; i < j; i++){ //loop B
    studentList[i].printDetails();
}
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Table 2-3. Final Driver Program

Modification	
Observation	The only modification I've made to this code was adding delete ptr; ptr = nullptr; Because from my observation, the program dynamically allocates a new Student object during every iteration of Loop A using the new operator. The flaw is that after the object;s data is copied into the corresponding element of the studentList array, the dynamically allocated object is no longer needed (it becomes a garbage value). This will result in a memory leak because that allocated memory will still remain in the heap that is why by adding delete ptr; it properly destroy the dynamically allocated to release memory back to the system and for good practice I added ptr = nullptr; to make sure it points to nothing.

```
for(int i = 0; i < j; i++){ //loop A
    Student *ptr = new Student(namesList[i], ageList[i]);
    studentList[i] = *ptr;
    delete ptr;
    ptr = nullptr;
}
```

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

```cpp
#include <iostream>
#include <string>

class Fruitz{
private:
    std::string Name;
    int Price;
    int Quantity;

public:
    // Constructor
    Fruitz(std::string name, int price, int quantity){
        Name = name;
        Price = price;
        Quantity = quantity;
    }

    // Copy Constructor
    Fruitz(const Fruitz& copy){
        Name = copy.Name;
        Price = copy.Price;
        Quantity = copy.Quantity;
    }

    // Destructor
    ~Fruitz(){
    }

    // Copy Assignment Operator
    Fruitz& operator=(const Fruitz& copy){
        if(this != &copy){
            Name = copy.Name;
            Price = copy.Price;
            Quantity = copy.Quantity;
        }
        return *this;
    }

    void displayAttributes() const{
        std::cout << Name << ":" << " PHP " << Price << " x" << Quantity << std::endl;
    }

    int getPrice() const{
        return Price;
    }

```

```
    int getQuantity() const{
        return Quantity;
    }
};

class Vegetablez{
private:
    std::string Name;
    int Price;
    int Quantity;

public:
    // Constructor
    Vegetablez(std::string name, int price, int quantity){
        Name = name;
        Price = price;
        Quantity = quantity;
    }

    // Copy Constructor
    Vegetablez(const Vegetablez& copy){
        Name = copy.Name;
        Price = copy.Price;
        Quantity = copy.Quantity;
    }

    // Destructor
    ~Vegetablez(){
    }

    // Copy Assignment Operator
    Vegetablez& operator=(const Vegetablez& copy){
        if(this != &copy){
            Name = copy.Name;
            Price = copy.Price;
            Quantity = copy.Quantity;
        }
        return *this;
    }

    void displayAttributes() const{
        std::cout << Name << ":" << " PHP " << Price << " x" << Quantity << std::endl;
    }
```

```

    int getPrice() const{
        return Price;
    }

    int getQuantity() const{
        return Quantity;
    }
};

```

Analysis:

For this problem, I created two separate classes, that being Fruitz and Vegetablez to be able to differentiate between fruits and vegetables. Both classes contain the same private attributes: Name, Price and Quantity to store the item's name, price and the quantity purchased. Each class contains the constructor, destructor, copy constructor, and assignment operator (the big 3 discussed from the previous ILO). In both of the classes I've also included a displayAttributes() function to display the item's detail, while getPrice() and getQuanitity() are getter functions to allow totalSum() function to access the object's data without directly accessing the private variables from both of the classes.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

```
int main(){

    const int F_SIZE = 2;
    const int V_SIZE = 2;

    Fruitz *fruitz[F_SIZE];
    Vegetablez *vegetablez[V_SIZE];

    fruitz[0] = new Fruitz("Apple", 10, 7);
    fruitz[1] = new Fruitz("Banana", 10, 8);

    vegetablez[0] = new Vegetablez("Broccoli", 60, 12);
    vegetablez[1] = new Vegetablez("Lettuce", 50, 10);

    std::cout << "Jenna's Grocery List" << std::endl;

    for(int i = 0; i < F_SIZE; i++){
        fruitz[i]->displayAttributes();
    }

    for(int i = 0; i < V_SIZE; i++){
        vegetablez[i]->displayAttributes();
    }
```

Analysis:
I created F_SIZE and V_SIZE, where they are declared as constants because the number of fruits and vegetables does not change throughout the program and it makes it easier to read. I created grocery items that were dynamically allocated instead of regular objects, using the new operator so I can be able to perform problem 4 later on, this iis stored in arrays of pointers named fruitz and vegetablez. Each pointer stores the address of an object created on the heap. After that it will display "Jenna's Grocery List" then loops through each array and calls the displayAttributes() function where I used the arrow operator (->) to display all of Jenna's grocery items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

```cpp
// Total Sum Function
void totalSum(Fruitz *fruitz[], int fruitzSize,
              Vegetablez *vegetablez[], int vegetablezSize)
{
    int total = 0;

    for(int i = 0; i < fruitzSize; i++){
        if(fruitz[i] != nullptr){
            total += fruitz[i]->getPrice() * fruitz[i]->getQuantity();
        }
    }

    for(int i = 0; i < vegetablezSize; i++){
        if(vegetablez[i] != nullptr){
            total += vegetablez[i]->getPrice() * vegetablez[i]->getQuantity();
        }
    }

    std::cout << "\nTotal Sum: PHP " << total << std::endl;
}
```

Analysis:
This function is outside both the classes that is why I had to use getters to be able to access the object's data from both Fruitz and Vegetablez classes. TotalSum() function is used to calculate the total amount that Jenna has to pay. The function accepts two arrays of pointersm one for the fruits and one for the vegetables together with their sizes that was obtained from our constant values (F_SIZE and V_SIZE). Inside the function we have two for loops which multiply each item's price by its quantity. Before that it has to check whether the pointer is not equal to nullptr before accessing the object (this prevents the program from trying to access memory that has already been deleted or anything that does not point to a valid object). The arrow operator is used once again because the arrays store pointers to dynamically allocated objects once it reaches the object, it calls getPrice() and the same applies with getQuanitity(). After all objects have been processed, the function displays the computed total.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

```
    // Delete Lettuce (Problem 4)
    delete vegetablez[1];
    vegetablez[1] = nullptr;

    // Deallocate remaining memory
    for(int i = 0; i < F_SIZE; i++){
        delete fruitz[i];
        fruitz[i] = nullptr;
    }

    delete vegetablez[0];
    vegetablez[0] = nullptr;

    return 0;
}
```

Analysis:
Hare, I am deleting the dynamically allocated memory Vegetablez object stored at index 1 of the vegetable array. Using the delete operator it will deallocate the memory occupied by the Lettuce object on the heap, preventing a memory leak. Afterwards, the pointer is assigned nullptr to avoid becoming a dangling pointer that points to an invalid memory location.

```
PS C:\Users\Justine>  & 'c:\Users\Justine\.vscode\extensions\ms-vscode.cpptools-1.32.2-win32
-MIEngine-Out-03fehehl.srr' '--stderr=Microsoft-MIEngine-Error-tjlcz4ok.rkz' '--pid=Microsof
Jenna's Grocery List
Apple: PHP 10 x7
Banana: PHP 10 x8
Broccoli: PHP 60 x12
Lettuce: PHP 50 x10

Total Sum: PHP 1370
PS C:\Users\Justine>
```

Problem 5: Discussion

This activity has shown how dynamic memory allocation, an array of pointers and the Rule of Three can be applied in C++. Two different classes have been used to differentiate between fruit and vegetable but both store the same kind of data. Groceries were dynamically allocated using the new operator and an array of pointers was utilized for that purpose, hence, giving the program the opportunity to practice with pointers and dynamic memory allocation. TotalSum() function has been designed separately to calculate the total sum of groceries by calculating multiplication of price and number of goods. The Lettuce object was finally deleted using delete operator and assigning nullptr value to the pointer of this object to free the memory safely.

Conclusion:

This activity was quite difficult for me to understand because I've done it with little understanding. The ILO A, however, gave me a heads-up on how the Rule of Three worked because it was focused more on analysis, so this gave me somewhat of a guideline for ILO C. The ILO C took me hours because I was trying to figure out how everything connects and what I needed to do for every part of the code. I watched many video tutorials from CodeBeauty, BroCode, and many more. I also studied and reviewed arrays, pointers, arrays of pointers, and how everything connects when using the Rule of Three.

Overall, I learned how the Rule of Three, pointers, arrays, and dynamic memory allocation work together and how to be able to implement this into my own code. Although the procedure and doing the actual code were very difficult and definitely challenging, it helped me better understand how each concept is connected through trial and a lot of errors. Despite my difficulties doing the task, I think I handled it fairly well, but I still need more practice with pointers and dynamic memory allocation and everything in general. I will continue reviewing and practicing these topics to improve my understanding of C++ programming so I'll be ready for more advanced topics in the future.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, summarization, code assistance and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used it to be able to understand what the code is exactly doing and to be able to explain it in a more detailed manner to be able to review it later for a better learning curve.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.